Integer Programming

Torben Thaysen

October 9, 2026

We examine the feasibility of n integer inequalities over $m \leq n$ variables of the form

$$\boldsymbol{A}x \leq b \tag{1}$$

with $\boldsymbol{A} \in \mathbb{Z}^{n \times m}$, $b \in \mathbb{Z}^n$, $x \in \mathbb{Z}^m$. We assume that $\boldsymbol{A}$ has full column rank m and that an initial basis is given. By Farkas' lemma, either the relaxation of the above to $x \in \mathbb{R}^m$ has a solution or there exists $y \in \mathbb{R}^{1 \times n}$ with $y \geq 0$ such that

$$y\boldsymbol{A} = 0 \qquad \text{and} \qquad yb < 0.$$

Such a y is a certificate of infeasibility for the relaxation and thus also for the integer problem. Our algorithm should thus either find an integral x satisfying (1) or produce such a certificate. To certify infeasibility of the integer problem, the Farkas certificate can include additional derived inequalities (branching bounds and cuts, see Section 2), which are themselves certified and added to the problem statement.

1 Solving the Relaxation

We implement a variant of the simplex method which minimizes the sum of infeasibilities. For a relaxed $x \in \mathbb{R}^m$ we write our problem as

$$\boldsymbol{A}x + p - s = b \qquad \text{with} \qquad p \geq 0, \quad s \geq 0 \quad \text{and} \quad s^{\mathsf{T}}p = 0.$$

We maintain a subset of m linearly independent rows of (1) for which equality holds, called the basis. The remaining rows are split based on the sign of $b - \boldsymbol{A}x$: rows with $b - \boldsymbol{A}x \geq 0$ form the second group and rows with $b - \boldsymbol{A}x < 0$ form the third group. Rows outside the basis for which equality happens to hold therefore belong to the second group. By reordering the rows, the matrix and vectors can be written as:

$$\boldsymbol{A} = \begin{pmatrix} \boldsymbol{A}_0 \\ \boldsymbol{A}_+ \\ \boldsymbol{A}_- \end{pmatrix} \qquad b = \begin{pmatrix} b_0 \\ b_+ \\ b_- \end{pmatrix} \qquad p = \begin{pmatrix} 0 \\ p_+ \\ 0 \end{pmatrix} \qquad s = \begin{pmatrix} 0 \\ 0 \\ s_- \end{pmatrix}$$

Using this decomposition the equations can be written separately as:

$$\boldsymbol{A}_0 x = b_0 \qquad \boldsymbol{A}_+ x + p_+ = b_+ \qquad \boldsymbol{A}_- x - s_- = b_-$$

Here $\boldsymbol{A}_0$ is an $m \times m$ square matrix which is invertible because the basis rows are linearly independent, such that $x = \boldsymbol{A}_0^{-1} b_0$. The second group of equations satisfies the desired

inequality $\boldsymbol{A}_+ x \leq b_+$, while the last group satisfies the opposite $\boldsymbol{A}_- x > b_-$, i.e. $s_- > 0$ componentwise. We thus have a satisfying solution iff the last group is empty.

For the pivot operation, assume that the leaving index (in the first group) is j. The equation for the first group is thus

$$\boldsymbol{A}_0 (x + \Delta x) + \lambda e_j = b_0,$$

where both positive and negative values are allowed for λ. From the invertibility of $\boldsymbol{A}_0$ and the previous value of x we find that

$$\Delta x = -\lambda \boldsymbol{A}_0^{-1} e_j.$$

The changes in p and s can be derived to be:

$$\Delta p_+ = -\boldsymbol{A}_+ \Delta x = \lambda \boldsymbol{A}_+ \boldsymbol{A}_0^{-1} e_j \qquad \Delta s_- = \boldsymbol{A}_- \Delta x = -\lambda \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j$$

After the pivot, row j is no longer part of the basis and its slack is λ. For $\lambda \geq 0$ it belongs to the second group with $p_j = \lambda$, while for $\lambda < 0$ it belongs to the third group with $s_j = -\lambda$. Since we want to reduce s to 0, we examine how $1^\mathsf{T} s$ changes:

$$1^\mathsf{T} \Delta s = \begin{cases} -1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j \cdot \lambda & \text{if } \lambda \geq 0 \\ -\left(1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j + 1\right) \cdot \lambda & \text{if } \lambda < 0 \end{cases}$$

So if $1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j > 0$, choosing a positive λ will decrease $1^\mathsf{T} s$. Similarly, when $1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j < -1$, progress can be made using a negative λ. But if $-1 \leq 1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} e_j \leq 0$, either sign of λ will result in no change or an increase in $1^\mathsf{T} s$. The choice of j among all indices which allow progress is called the pivot selection rule.

If no j allows progress, it is natural to ask if we can produce an infeasibility certificate. And indeed for $y = \begin{pmatrix} y_0 & y_+ & y_- \end{pmatrix}$ with

$$y_0 = -1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} \qquad y_+ = 0 \qquad y_- = 1^\mathsf{T}$$

we find that $y\boldsymbol{A} = 0$ and $yb < 0$ are both satisfied:

$$y\boldsymbol{A} = y_0 \boldsymbol{A}_0 + y_- \boldsymbol{A}_- = -1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} \boldsymbol{A}_0 + 1^\mathsf{T} \boldsymbol{A}_- = 0$$
$$yb = y_0 b_0 + y_- b_- = -1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} b_0 + 1^\mathsf{T} b_- = -1^\mathsf{T} \left(\boldsymbol{A}_- x - b_-\right) = -1^\mathsf{T} s_- < 0$$

where $1^\mathsf{T} s_- > 0$ holds because $s_- > 0$ componentwise, unless the third group is empty and the problem is already satisfied. Because $y_0 = -1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} \geq 0$ is just the no-progress condition for $\lambda \geq 0$ from above, it follows that $y \geq 0$. So y is a valid certificate.

Note that the certificate only requires the no-progress condition for $\lambda \geq 0$, while the condition $1^\mathsf{T} \boldsymbol{A}_- \boldsymbol{A}_0^{-1} \geq -1$ for pivots with $\lambda < 0$ is not needed. The algorithm may therefore stop with a certificate as soon as no pivot with $\lambda \geq 0$ is possible. Further, some empirical testing showed that including pivots with $\lambda < 0$ increased the number of pivots for all four tested pivot selection rules. The effect was especially strong for infeasible problems. We thus do not consider $\lambda < 0$ pivots for the initial implementation.

The above is only valid for values of λ where none of the components of p_+ or s_- change sign. Therefore λ is selected to be the value where the first component of p_+ or s_- becomes 0. This is the entering row and replaces the leaving row j in the basis. If several components become 0 for the same λ, any of them may be chosen as the entering row. Such a value of λ always exists because it is precisely chosen such that $1^\mathsf{T} \Delta s_- < 0$ and so at least one component of s_- decreases and eventually reaches 0.

2 Branch and Cut

The algorithm so far solves the relaxation for $x \in \mathbb{R}^m$. If the relaxation is infeasible, the integer problem is infeasible as well. But the algorithm can also terminate with a solution $x^\star \notin \mathbb{Z}^m$. In this case we derive a new constraint that is satisfied by all integral solutions but not by the current solution $x^\star$. There are two primary methods to do this: branching, which derives a new clause, and cutting, which derives a new inequality (a cutting plane). With branching we select some $x_i^\star \notin \mathbb{Z}$ and add the clause $x_i \leq \lfloor x_i^\star \rfloor \vee x_i \geq \lceil x_i^\star \rceil$ to the ambient SMT solver. By construction this clause is not satisfied by the current solution $x^\star$ and thus forces further updating.

For cutting we present a version of the Chvátal-Gomory cut that integrates well with the relaxation algorithm. Mixed integer cuts may be considered as an improvement in the future. Consider the expression

$$\omega(x) = \lfloor x^\star \rfloor - x - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor (b_0 - \boldsymbol{A}_0 x)$$

for arbitrary x and $x^\star = \boldsymbol{A}_0^{-1} b_0$ still denoting our (fixed) non-integer solution. For integer x, $\omega(x)$ is also an integer because $\boldsymbol{A}_0$ and b_0 are integral and the other terms are integral by construction. At the same time we may rewrite this as

$$\begin{aligned}
\omega(x) &= \lfloor x^\star \rfloor - x - \boldsymbol{A}_0^{-1} (b_0 - \boldsymbol{A}_0 x) + \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) (b_0 - \boldsymbol{A}_0 x) \\
&= \left(\lfloor x^\star \rfloor - \boldsymbol{A}_0^{-1} b_0\right) - \left(x - \boldsymbol{A}_0^{-1} \boldsymbol{A}_0 x\right) + \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) (b_0 - \boldsymbol{A}_0 x) \\
&= \left(\lfloor x^\star \rfloor - x^\star\right) + \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) (b_0 - \boldsymbol{A}_0 x) \,.
\end{aligned}$$

Here we can read off that $\omega(x) > -1$ for any feasible x because $b_0 - \boldsymbol{A}_0 x \geq 0$ holds for such x, the coefficients $\boldsymbol{A}_0^{-1} - \lfloor \boldsymbol{A}_0^{-1} \rfloor$ are non-negative by construction and $x^\star - \lfloor x^\star \rfloor < 1$ is also true by construction.

Combining the properties, we deduce that for feasible integral x, $\omega(x)$ satisfies both $\omega(x) \in \mathbb{Z}^m$ and $\omega(x) > -1$, which implies that $\omega(x) \geq 0$. The existing non-integral solution, on the other hand, has

$$\omega(x^\star) = \left(\lfloor x^\star \rfloor - x^\star\right) + \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) (b_0 - \boldsymbol{A}_0 x^\star) = \lfloor x^\star \rfloor - x^\star$$

because $\boldsymbol{A}_0 x^\star = b_0$. By choosing some $x_i^\star \notin \mathbb{Z}$, we have that $\omega_i(x^\star) = \lfloor x_i^\star \rfloor - x_i^\star < 0$ but $\omega_i(x) \geq 0$ for all feasible integral x. By adding $\omega_i(x) \geq 0$ to the problem statement we make the solution $x^\star$ infeasible and continue the search. More precisely, the new inequality will land in the third group and the search can continue without restarting.

Written out in the form of the problem statement, the inequality $\omega(x) \geq 0$ is

$$\begin{aligned}
\left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) \boldsymbol{A}_0 x &\leq \left(\lfloor x^\star \rfloor - x^\star\right) + \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) b_0 \\
&= \left\lfloor \boldsymbol{A}_0^{-1} b_0 \right\rfloor - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor b_0 \\
&= \left\lfloor \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) b_0 \right\rfloor .
\end{aligned}$$

With $u = e_i^\mathsf{T} \left(\boldsymbol{A}_0^{-1} - \lfloor \boldsymbol{A}_0^{-1} \rfloor\right)$ the cut $\omega_i(x) = e_i^\mathsf{T} \omega(x) \geq 0$ is of the form

$$u \boldsymbol{A}_0 x \leq \lfloor u b_0 \rfloor \,. \tag{2}$$

This is useful since u provides a certificate for the cut and also shows that the cut only depends on the rows where $u_j \neq 0$.

In fact, the form (2) provides its own validity proof of the cut. Since $u \geq 0$ by construction, the non-negative combination of inequalities $u\boldsymbol{A}_0 x \leq ub_0$ follows from the original inequalities. At the same time, the left-hand side vector

$$u\boldsymbol{A}_0 = e_i^\mathsf{T} \left(\boldsymbol{A}_0^{-1} - \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor\right) \boldsymbol{A}_0 = e_i^\mathsf{T} - e_i^\mathsf{T} \left\lfloor \boldsymbol{A}_0^{-1} \right\rfloor \boldsymbol{A}_0$$

is a combination of integral terms and thus itself integral. Therefore, for integer x, the right-hand side ub_0, which is not necessarily an integer, can be safely rounded down, giving back (2).

3 Implementation

The implementation should use sparse matrix and vector representation. The numbers should be stored as signed fixed-point integers with variable size, accomplished by templating all linear-algebra-related classes on the fixed-point layout. The layout is fixed per run and can only be changed by a restart. The representation can increase separately in magnitude and precision in multiples of 32 bits when too large or too small numbers are encountered. The baseline for a sparse entry should be 128 bits with a 32-bit column/row index, 64 bits before the point and 32 bits after it. A slight variation may be considered to represent unsigned 64-bit values in the baseline representation.

Matrices are stored as a list of lists of entries in both row and column major order as needed. For example, the LU decomposition of $\boldsymbol{A}_0$ needs both to efficiently support left and right solves. Without this, for either left or right multiplication a sparse-sparse dot product would have to be computed per column or row, forcing $O(\text{columns})$ complexity as opposed to $O(\text{entries})$. Where possible matrix and vector operations should be implemented using an optimized out-of-place add and multiply-and-add primitive from a previous project. The output vector size is allocated as the sum of the two input sizes, which is a basic worst-case estimate. Later, allocation should be optimized with a local arena allocator for the vectors.

Error bounding and validation techniques should be used to tightly control the error and ensure soundness.

4 Useful Bounds

4.1 Lower bound on non-integrality

In the presence of finite precision the exact solution x of $\boldsymbol{A}_0 x = b_0$ and its approximation $\tilde{x}$ will be different. When x is integral it will be equal to the rounded solution $x_r = \text{round}(\tilde{x})$ as long as the absolute error is bounded componentwise by $|x - \tilde{x}| < \frac{1}{2}$, which is easy. To detect if a close to integral solution is a genuine integer solution we can thus test if $\boldsymbol{A}_0 x_r = b_0$ with exact integer arithmetic. The issue is that when this fails we need to find a non-integral component of the solution to perform a branch or cut, which is difficult when all components of $\tilde{x}$ are within the margin of error of an integer. This can only be resolved by reducing error (typically by increasing precision) until a component of $\tilde{x}$ separates from its integer neighbour. A basic lower bound on how close $\tilde{x}$ can be to an

integer follows from the exact expression

$$x = \boldsymbol{A}_0^{-1} b_0 = \frac{\operatorname{adj}(\boldsymbol{A}_0) b_0}{\det(\boldsymbol{A}_0)}.$$

Since the adjugate of an integer matrix is again an integer matrix, the components of x can always be written as rational numbers with $\det(\boldsymbol{A}_0)$ as the denominator. Therefore the closest any component of x can be to an integer without being one is $1/\left|\det(\boldsymbol{A}_0)\right|$.

This is quite a weak bound, so we also prove the following result:

Lemma 4.1. *In terms of the residual $r = b_0 - \boldsymbol{A}_0 x_r \in \mathbb{Z}^m$ we have that*

$$\left\|x - \tilde{x}\right\|_\infty + \left\|\tilde{x} - x_r\right\|_\infty \geq \left\|x - x_r\right\|_\infty \geq \max_i \frac{|r_i|}{\left\|a_i\right\|_1}.$$

This is useful since in our use case we have $\boldsymbol{A}_0 x_r \neq b_0$ and thus $r \neq 0$. Combining this with $r \in \mathbb{Z}^m$ gives $|r_i| \geq 1$ for some i and a lower bound of the form $1/\left\|a_i\right\|_1$ on the largest deviation which is often much stronger than the $1/\left|\det(\boldsymbol{A}_0)\right|$ bound.

Proof. The first inequality is simply the triangle inequality. For the second inequality write r as

$$r = b_0 - \boldsymbol{A}_0 x_r = \boldsymbol{A}_0 \left(x - x_r\right)$$

and observe the inequality chain

$$\begin{aligned} |r_i| &= \left|a_i \left(x - x_r\right)\right| \\ &\leq \left|a_i\right| \left|x - x_r\right| \\ &\leq \left\|a_i\right\|_1 \left\|x - x_r\right\|_\infty. \end{aligned}$$

The bound then follows by dividing by $\left\|a_i\right\|_1$ and taking the maximum over i. □